

Packet injector testing report

Alireza Monemi

December 10, 2019

1 Fixed bugs in packet injector

We have noticed that the following bugs have been fixed by SemiDynamics in the current version of the packet injector:

- The old version of the packet injector did not respond to any snoop requests sent by the NoC. We have noticed that this bug is fixed in the current version.
- The bugs in CHI-interface including connection and Struct size that we have reported before have been fixed.

2 Test-case simulation configuration

The simulation model is generated using one packet injector from SemiDynamics, three basic request nodes (RNF), four home-nodes (HNF) and one slave node (SNF) connected to a 3×3 Mesh NoC as shown in Figure 1.

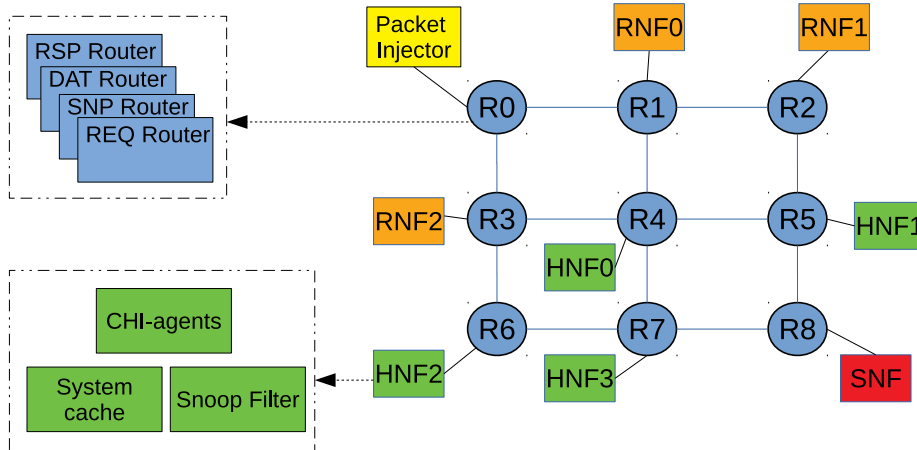


Figure 1: The simulation model functional block diagram used for testing the packet injector.

The NoC is equipped with a distributed system cache and snoop filter. In the test-case model, the address space is divided into four partitions where consecutive cache block addresses go to different home nodes. The destination home node index number is obtained from the following equation:

$$n = (address/64)\%H \quad (1)$$

where n is the destination HNF index, 64 is the cache block size in bytes, and H is the maximum number of home nodes in the system (4 in this configuration).

3 The passed test-case scenarios

The current version of packet injector passes the following test-case scenarios:

3.1 TestCase #1

- **Configuration setting:** The packet injector is the only active request node in the system. It reads, three application-driven trace files extracted by Gem 5 generated using MB2020 D4.4-SVE trace generator (will be called APT-trace file at the rest of the report). The trace files have 25, 1279, and 13310 normal (not exclusive) Load Miss traces, respectively.
- **Observation:** Simulation using trace files including 25 and 1279 results in only `ReadShared` transactions in the NoC.

Simulation using 13310 load-miss results in some `SnpCleanInvalid` transactions that are generated by home nodes due to snoop-filter evictions. Some `SnpCleanInvalid` requests are responded by `DAT_SnpRespData_PD` with data filed set to 0. For those responses, the home node updates the main memory as it got the pass dirty data.

3.2 TestCase #2

- **Configuration setting:** The packet injector is the only active request node in the system. It reads custom generated trace files sending Load-miss to linearly increasing cache address blocks. The generated trace file has the following format:

```
0:  0 0 MISS 0 0x0 D LD
1:  0 0 WAIT 10 0
2:  0 0 MISS 1 0x40 D LD
3:  0 0 WAIT 10 0
.
.
n:  0 0 MISS 2 (n/2)*0x40 D LD
n+1:0 0 WAIT 10 0
```

The cache block address size is 64 bytes (0x40). The trace file contains 600 load-miss in total.

- **Observation:** The simulation is run successfully with no snoop filter eviction.

4 The failed test case scenarios

4.1 TestCase #3

- **Configuration setting:** Packet injector is the only active request node in the system. It reads an APT-trace file having the combination of normal and exclusive Load Miss traces:

```
0: 0 0 MISS 0 0x811ec000 D LD X
1: 7687238951 0 MISS 1 0x811ec000 D LD X
2: 7687238965 0 WAIT 14 4282045465 0
3: 7687238965 0 MISS 2 0x88208000 D ST E
4: 0 0 MISS 0 0x811ec000 D ST X
5: 0 0 MISS 0 0x811ec000 D LD X
6: 0 0 MISS 0 0x811ec000 D ST X
7: 0 0 MISS 0 0x811ec000 D LD X
8: 0 0 MISS 0 0x811ec000 D ST X
9: 0 0 MISS 0 0x811ec000 D LD X
10: 0 0 MISS 0 0x811ec000 D ST X
11: 0 0 MISS 0 0x811ec000 D LD X
12: 0 0 MISS 0 0x811ec000 D ST X
13: 0 0 MISS 0 0x811ec000 D LD X
14: 0 0 MISS 0 0x811ec000 D ST X
15: 0 0 MISS 0 0x811ec000 D LD X
16: 0 0 MISS 0 0x811ec000 D ST X
17: 0 0 MISS 0 0x811ec000 D LD X
18: 0 0 MISS 0 0x811ec000 D ST X
19: 0 0 MISS 0 0x811ec000 D LD X
20: 0 0 MISS 0 0x811ec000 D ST X
21: 0 0 MISS 0 0x811ec000 D LD X
22: 0 0 MISS 0 0x811ec000 D ST X
23: 0 0 MISS 0 0x811ec000 D LD X
24: 0 0 MISS 0 0x811ec000 D ST X
25: 0 0 MISS 0 0x811ec000 D LD X
26: 0 0 MISS 0 0x811ec000 D ST X
28: 7687239208 0 WAIT 243 0 0
29: 7687239208 0 WAITFOR 1 0x811ec000
30: 7687239208 0 MISS 3 0x811ec040 D LD X
31: 7687239221 0 WAIT 13 0 0
32: 7687239221 0 MISS 4 0xfe808040 D ST E
```

```

33: 0 0 MISS 0 0x811ec040 D ST X
34: 0 0 MISS 0 0x811ec040 D LD X
.
.

```

- **Observation:** The packet injector only generates the first two **ReadShared** Exclusive transactions to the same cache address. Then it stops reading further traces. All of the two transactions are done completely (ACK responses have been received from packet injector for both transactions), and the snoop filter status is free at the end of the simulation.

4.2 TestCase #4

- **Configuration setting:** The packet injector is the only active request node in the system. It reads a custom generated trace file sending 600 Load-misses where all the load misses are pointed to the same row of SnoopFilter cache memory. The snoop-filter uses 8 ways set-associative cache. So in this test-case scenario it is expected to get $600-8=592$ evictions (**SnpCleanInvalid** transactions) in the simulation log file. The following is the trace file format:

```

0: 0 0 MISS 0 0x0 D LD
1: 0 0 WAIT 10 0
2: 0 0 MISS 1 0x10000 D LD
3: 0 0 WAIT 10 0
.
.
n: 0 0 MISS 2 (n/2)*0x10000 D LD
n+1:0 0 WAIT 10 0

```

The trace file has a similar format with test-case#2 with the difference that the address increases in each line by $1024*64$ for each load-miss. Based on equation (1) all requests are sent to HNF_0 . The cache address index width of snoop filter is set to 10-bit, so the status of all addresses are kept in the first row of the snoop filter (index=0).

- **Observation:** The simulation is not completed as the packet injector did not read the whole of the trace file (600 load-miss). By checking the simulation transcript file we noticed that the packet injector has sent only 19 **ReadShared** requests. Moreover, all of these transactions have been completed as there was no active transaction in the home node. Total of 11 evictions is found in transcript file, which matches the expectation for the 19 load miss to the same row of the snoop filter.

5 Suggestions on DCT simulation

We have tried to generate a transaction that results in the occurrence of direct cache transfer (DCT) in the system. Our plan was to ask the packet injector to directly transfer data to another request node.

5.1 TestCase #5

- **Configuration setting:** there are two active request nodes in the system. The packet injector and one basic RNF (RNF0). The packet injector reads trace files including normal load misses. After this transaction is done successfully, RNF0 sends a **ReadShared** request to the same address as packet injector sent before. The home node checks its snoop filter and realizes that this cache block may be valid inside the packet injector in a unique state and decide to establish a DCT. As the packet injector got a valid copy of that cache line before, we expected that the requested cache line may be in one of the Unique Clean (UC) or the Unique Dirty (UD) states. Hence, the expected transaction flow for this test-case is one of the two transaction flows shown in Figures 2 and 3. (RN-F1 represents the packet injector in these figures):

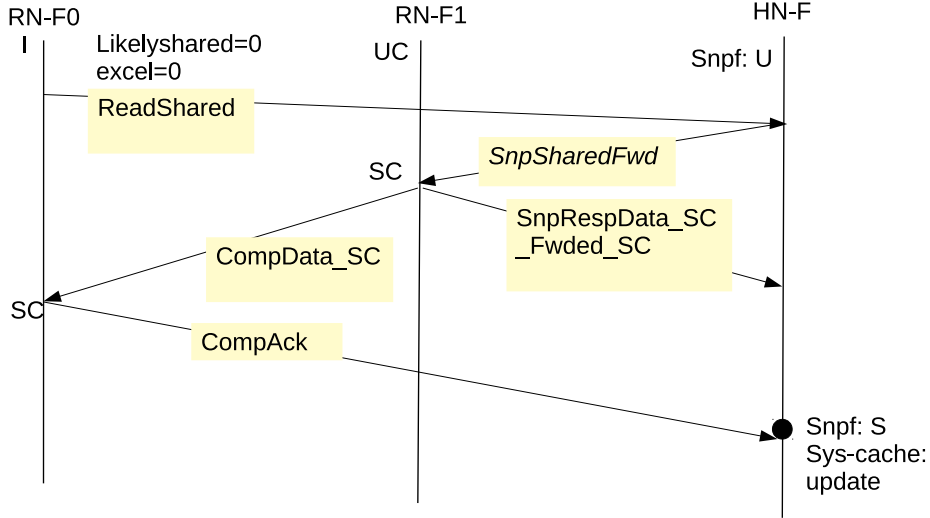


Figure 2: DCT, SNPF status is Unique, the cache block in packet injector is in clean and unique state

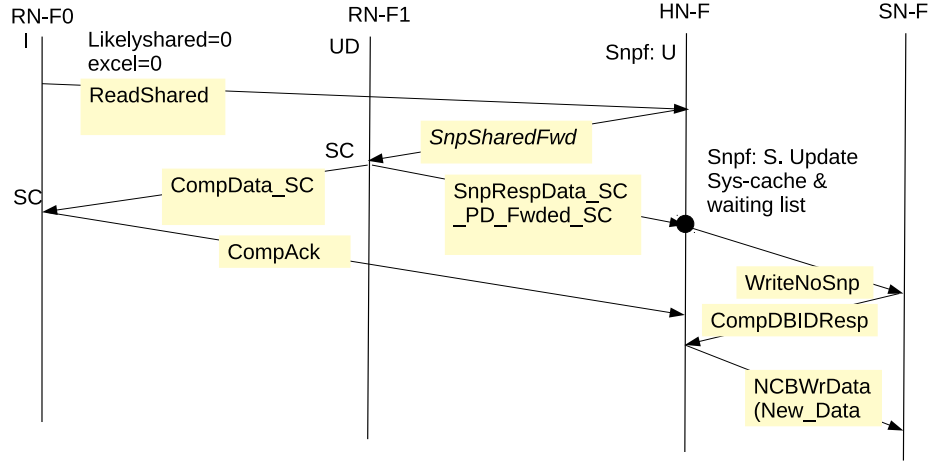


Figure 3: DCT, SNPF status is Unique, the cache block in packet injector is in dirty and unique state.

However, the packet injector responses with **SnpResp_I**. As a result, the following transaction flow happens in the simulation model:

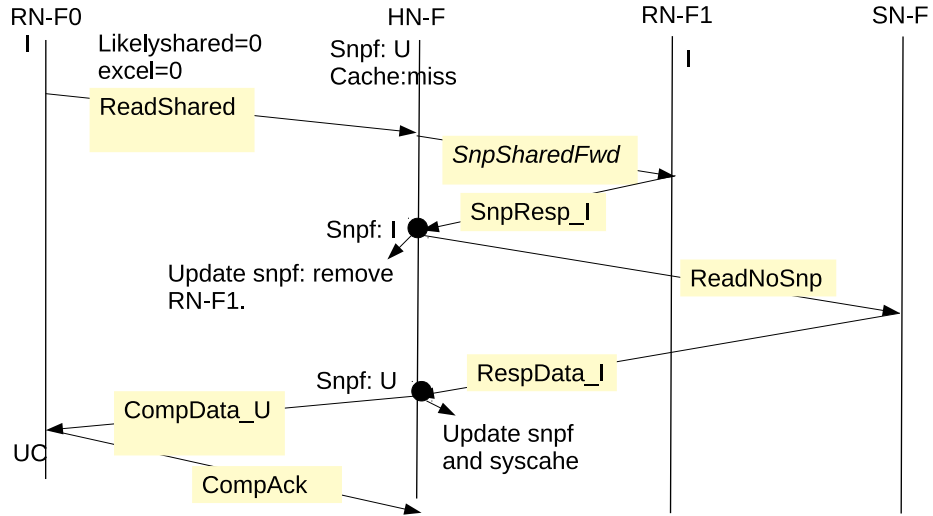


Figure 4: DCT, SNPF status is Unique, the cache block in packet injector is in Invalid.

Although, **SnpResp_I** is a valid response from the packet injector as it shows a silent cache transition in the packet injector without notifying the home node. However, it prevents us to model or analyze any type of DCT transaction flows in a system in the future.

We are aware of this fact that the packet injector works only on traces and it does not keep the actual data. However, it may be possible to send a data packet with zero/X data filed (similar to what I observe in Test Case #1 in the large trace file simulation). This will allow us to model a DCT transaction using the packet injector in the future.

6 Questions

1. We have noticed that **Size** (shows the atomic instruction width) filed has been added to atomic **struct** by checking the diff version of new and old packet injector source files. However, the **Size** filed is still missing in request flit. In our home, node this filed is needed to take the atomic instruction width and it affects the way Data Processing Unit executes the atomic instruction. Is there another field that represent the atomic instruction width or the **Size** filed should be added to the request flit?
2. We also do not know how to generate files that helps SemiDynamics to test similar test-case failure scenarios in their simulation environment. As in the failed test cases, the packet injector gets some CHI snoop requests that depend not only on the read trace file but also depend on the NoC configuration and other request nodes trace files. We would like to know if the SemiDynamics' CHI agent is also capable of reading a trace file? How the **SnpCleanInvalid** transaction is tested in your simulation model?

7 Plans for future work

- We have tested our own developed request node under several stress-test case scenarios. The test-case#4 is one example of those test cases. We plan to test the packet injector under similar stress test case scenarios as well.
- Our next plan is also to have multiple active packet injectors in one system where each read different trace files.